CHEAT SHEET · JS INTERVIEW #06 @specsycoder

0.1 + 0.2 !== 0.3

Why JavaScript gets this "wrong", and how to fix it

float.js

```
0.1 + 0.2; // 0.30000000000000004
0.1 + 0.2 === 0.3; // false
```

output `false` `0.30000000000000004`

Why it happens

01 Binary
Computers store numbers as 0s and 1s.

02 Never ends
0.1 in binary repeats forever, like 1 ÷ 3 = 0.3333... in decimal.

03 Rounded
JS keeps the closest 64-bit value (IEEE 754), so 0.1 is stored slightly off.

04 Errors add up
Two tiny errors together show up as extra digits.

How to fix it

fix.js

```
// money: work in paise, not rupees
10 + 20 === 30; // true
// other numbers: allow a tiny gap
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON; // true
```

Save this · share with a friend Follow @specsycoder for daily JS interview Qs

@specsycoder

Pick the right fix

What to use, and where

situation	use
money math	integers (paise), convert only to display
equality check	`Math.abs(a - b) < Number.EPSILON`
showing to users	`toFixed(2)` or `Intl.NumberFormat`
very large / exact values	`BigInt` or a decimal library

edge.js

```
(0.1 + 0.2).toFixed(2); // "0.30" (a string)
0.5 + 0.25 === 0.75; // true, exact
(1.005).toFixed(2); // "1.00", not "1.01"
Number.EPSILON; // 2.22e-16
```

Traps to mention

- **Not every decimal is off.** 0.5 and 0.25 are exact, because they are powers of 2.
- **EPSILON suits numbers near 1.** For big values, scale the tolerance to the size of the numbers.
- **toFixed returns a string** and can round unexpectedly, like `1.005`.

Your 20-second answer

"JavaScript stores numbers as 64-bit binary floating point. 0.1 and 0.2 cannot be stored exactly in binary, so each is slightly off, and the sum becomes 0.30000000000000004. For money I use integers like paise, and for comparisons I check that the difference is smaller than a tiny tolerance like Number.EPSILON."